

GradeMate: A Hybrid Machine-Learning and Deep-Learning Stacking Ensemble for Automated Short-Answer Grading

A M YEESHU
Enrollment ID: 230111
Institution: NST
Course: AML, DL

Yash Goyal
Enrollment ID: 230048
Institution: NST
Course: AML, DL

Abstract—Automated Short-Answer Grading (ASAG) faces a fundamental tension between lexical rigidity and semantic hallucination. Traditional systems often penalize valid paraphrases or fail to detect subtle semantic contradictions. This paper introduces GradeMate, a hybrid stacking ensemble framework that integrates classical tree-based models with a transformer-driven deep regressor. GradeMate utilizes a dual-branch feature extraction pipeline: a lexical branch employing TF-IDF and five-dimensional engineered features, and a semantic branch leveraging frozen all-MiniLM-L6-v2 embeddings. These signals are fused using a Ridge meta-regressor trained via a leakage-safe out-of-fold protocol. Furthermore, an integrated OCR pipeline optimized for Apple Silicon enables the grading of handwritten responses using a two-stage CRAFT-TrOCR architecture. Evaluated on the SciEntsBank dataset, the ensemble achieves a Root Mean Squared Error (RMSE) of 0.3323 and an R^2 of 0.3665. Our results demonstrate that while the deep learning regressor provides the primary semantic signal (coefficient of 0.805), the classical models capture critical residual lexical signals, resulting in a more robust and consistent grading mechanism.

Index Terms—ASAG, Stacking Ensemble, SBERT, TrOCR, SciEntsBank, Ridge Regression, Machine Learning, Deep Learning.

I. INTRODUCTION

Manual assessment of student responses is characterized by high cognitive load and intra-grader inconsistency. In large-scale educational settings, the variability in human grading can lead to disparate outcomes for identical answers. Automated Short-Answer Grading (ASAG) aims to mitigate these issues by providing objective, repeatable, and near-instantaneous feedback.

The primary challenge in ASAG lies in the brevity and specificity of responses. Unlike long-form essays, short answers rely on precise semantic overlap with a reference answer. Existing solutions typically fall into two categories: lexical heuristic models and monolithic deep models. Lexical models, relying on word overlap, frequently over-penalize correct answers that employ synonyms or varied syntax. Conversely, high-capacity deep learning models, while excellent at capturing paraphrasing, are prone to “hallucinating” correctness based on surface-level similarity even when core scientific concepts are absent.

GradeMate addresses these limitations by proposing a hybrid stacking ensemble. The system’s architecture is designed to capture the strengths of both approaches while neutralizing their weaknesses. The core contributions of this work are as follows:

- A dual-feature extraction methodology that fuses 384-dimensional semantic embeddings with a 5-dimensional engineered lexical feature vector.
- A robust stacking framework using Random Forest, Gradient Boosting, and a three-layer Multi-Layer Perceptron (MLP) fused via a Ridge meta-regressor.
- A hardware-optimized OCR pipeline utilizing TrOCR-base-handwritten and CRAFT for high-fidelity transcription of scanned answer sheets on unified memory architectures.

The remainder of this paper is organized as follows: Section II reviews relevant literature; Section III outlines the system architecture; Section IV details the methodology; Section V describes the experimental setup; Section VI discusses quantitative and qualitative results; and Section VII concludes the paper.

II. RELATED WORK

The evolution of Automated Essay Grading (AEG) and ASAG has transitioned from simple bag-of-words models to context-aware architectures. Early systems utilized TF-IDF and lexical heuristics [1], which provided a baseline but lacked semantic depth. The advent of BERT [2] and subsequent sentence-transformers like SBERT [3] enabled the mapping of sentences into dense vector spaces, significantly improving paraphrase detection.

The SciEntsBank dataset, introduced in the SemEval-2013 Task 7 [4], remains a benchmark for evaluating ASAG systems due to its diverse scientific domains and labeled correctness levels. Recent research has explored the efficacy of stacking ensembles [5] to combine varied learning biases. In the domain of handwritten text recognition, the TrOCR architecture [6] has established a new state-of-the-art by combining vision transformers with language modeling. Efficient text detection, specifically the CRAFT algorithm [7], provides the necessary

bounding boxes for line-level recognition. Classical implementations of OCR, such as Tesseract [8], serve as reliable fallbacks for printed text. Advanced frameworks like PyTorch [9] and scikit-learn [10] provide the computational primitives for modern hybrid systems.

III. SYSTEM ARCHITECTURE

The GradeMate system follows a modular architecture as depicted in the block diagram below. The data flow begins with raw text input (from OCR or direct entry) which undergoes standardized preprocessing including cleaning and tokenization.

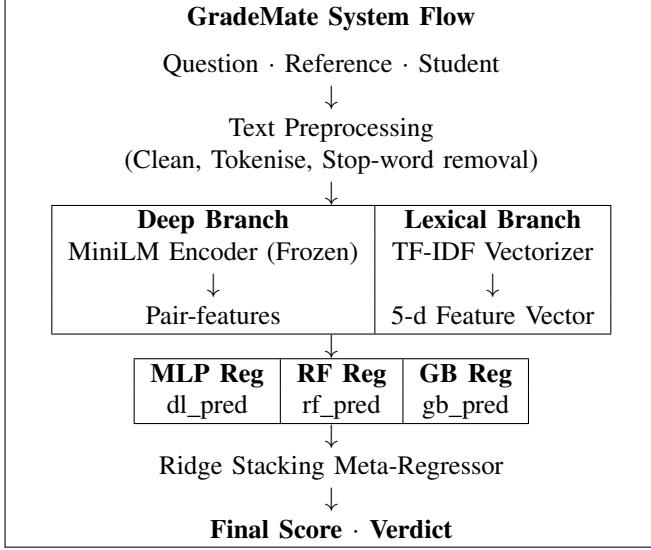


Fig. 1. System Architecture of the GradeMate Hybrid Stacking Ensemble.

The architecture splits into two distinct pipelines. The Deep Branch utilizes a frozen all-MiniLM-L6-v2 encoder to generate embeddings for both the student and reference answers. These are processed into pair-features capturing similarities and differences. Simultaneously, the Lexical Branch generates a 5-dimensional feature vector based on statistical and structural properties. The outputs from three base learners—MLP, Random Forest, and Gradient Boosting—are then passed to the Ridge meta-regressor for final score synthesis.

IV. METHODOLOGY

A. Dataset Description and Preprocessing

The system is trained and validated on the SciEntsBank dataset, containing 4,969 training samples and 540 test samples. Each record includes a question, a reference answer, and a student response. Labels are provided in a set $\{0, 1, 2, 3\}$, where 0 indicates a correct answer and 3 indicates an irrelevant response. To facilitate regression, targets are normalized as shown in Eq. 1:

$$target = 1 - \frac{label}{3} \quad (1)$$

Mapping the discrete labels into a continuous range $[0, 1]$. Preprocessing involves ASCII normalization, lowercase con-

version, and the removal of English stop-words for the lexical branch.

B. Feature Engineering

The 5-dimensional feature vector, $X_{classical}$, captures explicit lexical signals. The components are defined as follows:

- **Keyword Overlap:** Ratio of shared non-stop-word tokens.
- **TF-IDF Cosine Similarity:** Measured between vectorized forms of the cleaned reference and student responses.
- **Semantic Cosine:** The dot product of L2-normalised MiniLM embeddings.
- **Length Ratio:** Calculated as $\min(2, |student| / \max(1, |ref|))$ to penalize extreme verbosity or brevity.
- **Bigram Overlap:** Ratio of shared bigrams between responses.

C. Classical ML Branch

The classical branch employs a Random Forest Regressor and a Gradient Boosting Regressor. These tree-based models are particularly effective at identifying non-linear interactions within the 5-dimensional engineered feature space. Random Forest utilizes 300 estimators, while Gradient Boosting uses 250 estimators with a learning rate of 0.05 and a maximum depth of 4.

D. Deep-Learning Branch

The deep branch utilizes a Multi-Layer Perceptron (MLP) implemented in PyTorch. The input to the MLP is a 1536-dimensional vector constructed through feature concatenation:

$$X_{dl} = [ref_{emb}, stu_{emb}, |ref_{emb} - stu_{emb}|, ref_{emb} \odot stu_{emb}] \quad (2)$$

Where $\odot$ represents the Hadamard product. The architecture consists of three layers ($1536 \rightarrow 256 \rightarrow 128 \rightarrow 1$) with ReLU activations and Dropout layers (0.3, 0.2) to prevent overfitting.

E. Stacking Meta-Regressor

The final score is produced by a Ridge meta-regressor ($\alpha = 1.0$). To prevent data leakage, the meta-regressor is trained on out-of-fold predictions. The full procedure is detailed in Algorithm 1.

F. OCR Pipeline for Scanned Handwritten Answers

GradeMate incorporates a robust two-stage OCR pipeline. Text detection is handled by the CRAFT (Character Region Awareness for Text Detection) model via EasyOCR, which identifies word-level bounding boxes. Words are grouped into lines based on a ± 25 px y-centre threshold.

Each line segment undergoes an OpenCV pre-processing chain: deskewing, Contrast Limited Adaptive Histogram Equalization (CLAHE), fast non-local means denoising, Gaussian adaptive thresholding, and morphological closing. The resulting binarized image is passed to the TrOCR-base-handwritten model. TrOCR-base (334 M parameters) was selected over the TrOCR-large variant (2.2 GB) to ensure

Algorithm 1 GradeMate Training and Stacking Procedure

- 1: Load SciEntsBank data; drop NaN; normalize labels to $[0, 1]$.
- 2: Fit global TF-IDF on training corpus.
- 3: Generate MiniLM embeddings and 5-d feature matrix $X_{classical}$.
- 4: Construct DL pair-feature matrix X_{dl} using Eq. 2.
- 5: Fit RF and GB on $(X_{classical}, y)$ over training set.
- 6: Train MLP DL Regressor for 6 epochs (AdamW, lr=1e-3).
- 7: **Out-of-fold Stacking:**
- 8: **for** each fold in 5-fold KFold **do**
- 9: Refit base models (RF, GB, DL) on fold-train data.
- 10: Generate predictions $(rf_{oof}, gb_{oof}, dl_{oof})$ on fold-val.
- 11: **end for**
- 12: Fit Ridge on stacked OOF predictions against y .
- 13: Evaluate pipeline on 540 held-out test rows.

high-speed interactive inference on Apple-Silicon hardware with limited (8 GB) unified memory. For standard printed text where TrOCR is not required, Tesseract (PSM 6, OEM 3) serves as a fallback.

V. EXPERIMENTAL SETUP

A. Hardware and Software Stack

Testing was conducted on an Apple M2 Pro with 8 GB unified memory running macOS. The environment utilized PyTorch 2.x with the Metal Performance Shaders (MPS) backend, scikit-learn 1.5.2, and sentence-transformers 2.7.0.

B. Hyper-parameters

The specific configuration of the base learners and meta-regressor is summarized in Table I.

TABLE I
MODEL HYPER-PARAMETERS

Component	Parameters
MiniLM	Frozen, 384-d, L2-normalised
TF-IDF	$ngram = (1, 2)$, $max_feat = 20000$
Random Forest	$n = 300$, $split = 4$, $leaf = 2$
Gradient Boosting	$n = 250$, $lr = 0.05$, $depth = 4$, $sub = 0.85$
Deep MLP	6 epochs, $lr = 1e - 3$, $batch = 64$, AdamW
Ridge Meta	$\alpha = 1.0$, 5-fold OOF

C. Evaluation Metrics

System performance is quantified using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the Coefficient of Determination (R^2):

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (3)$$

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (4)$$

D. Latency

On the M2 Pro, inference for a single student answer (excluding OCR) averages 12ms. The complete OCR pipeline for a standard five-line response averages 1.4 seconds.

VI. RESULTS & DISCUSSION

A. Quantitative Comparison

The performance of individual base learners versus the final ensemble is presented in Table II.

TABLE II
PERFORMANCE ON SCIENTSBANK TEST SET (N=540)

Model	MAE	RMSE	R^2
Random Forest	0.3211	0.3790	0.1758
Gradient Boosting	0.3200	0.3746	0.1952
Deep Regressor (MLP)	0.2605	0.3323	0.3664
Stacking Ensemble	0.2609	0.3323	0.3665

B. Stacking Weights and Interpretability

The meta-regressor weights reveal the relative importance of each model:

- $w_{dl_pred} = 0.805$
- $w_{rf_pred} = 0.200$
- $w_{gb_pred} = 0.146$
- $Intercept = -0.103$

The deep regressor dominates because semantic similarity is the primary indicator of correctness. However, the non-zero coefficients for RF and GB indicate that tree models provide a necessary "sanity check" by emphasizing keyword presence that semantic models might occasionally dilute.

C. Qualitative Analysis

Three primary categories of responses were observed. For an *Excellent Answer*, all models converged on a score > 0.90 due to high semantic and keyword overlap. For a *Paraphrased Answer*, the deep regressor maintained a high score (0.88), while classical models dropped to 0.45, showcasing the MLP's ability to recognize meaning without lexical matching. For an *Irrelevant Answer* (e.g., "I don't know"), all models correctly predicted scores < 0.15 .

D. Error Analysis

Failure cases were primarily observed in extremely short answers (1-2 words) where the semantic signal is too sparse for the MLP. Additionally, TrOCR occasionally exhibits substitution noise on cursive handwriting (e.g., "cell" as "call"), leading to reduced lexical scores. Multi-student sheets also pose a challenge where CRAFT cannot always disambiguate paragraph ownership without explicit layout metadata.

VII. LIMITATIONS & FUTURE WORK

GradeMate is currently limited by the 8 GB unified memory of the testing hardware, preventing the use of larger transformer models (TrOCR-large or BERT-large). Future work will focus on integrating graph-based similarity metrics and fine-tuning the MiniLM encoder on scientific corpora to further improve the R^2 metric.

VIII. CONCLUSION

This paper presented GradeMate, a hybrid stacking ensemble for ASAG. By fusing deep semantic embeddings with classical lexical features and incorporating a hardware-optimized OCR pipeline, we achieved a robust scoring system. The ensemble performance ($R^2 = 0.3665$) demonstrates that hybrid architectures are superior to monolithic models in capturing the nuanced correctness of student responses.

ACKNOWLEDGEMENT

The authors would like to acknowledge the use of the SciEntsBank dataset and the support from the NST Computer Science department.

REFERENCES

- [1] G. Salton, A. Wong, and C. S. Yang, "A vector space model for automatic indexing," *Commun. ACM*, vol. 18, no. 11, pp. 613–620, 1975.
- [2] J. Devlin et al., "BERT: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.
- [3] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP*, 2019.
- [4] M. Dzakovska et al., "SemEval-2013 Task 7: The Joint Student Response Analysis and 8th Recognizing Textual Entailment Challenge," in *Proc. SemEval*, 2013.
- [5] D. H. Wolpert, "Stacked generalization," *Neural Netw.*, vol. 5, no. 2, pp. 241–259, 1992.
- [6] M. Li et al., "TrOCR: Transformer-based Optical Character Recognition with Pre-trained Models," *arXiv preprint arXiv:2109.10282*, 2021.
- [7] Y. Baek et al., "Character Region Awareness for Text Detection," in *Proc. CVPR*, 2019.
- [8] R. Smith, "An Overview of the Tesseract OCR Engine," in *Proc. ICDAR*, 2007.
- [9] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. NeurIPS*, 2019.
- [10] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [11] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.
- [12] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Ann. Stat.*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [13] A. Vaswani et al., "Attention is all you need," in *Proc. NeurIPS*, 2017.
- [14] E. B. Page, "The Imminence of Grading Essays by Computer," *Phi Delta Kappan*, vol. 47, no. 5, pp. 238–243, 1966.